Parallel Bigram Counting using OpenMP

by Marcos Fernández Hernández
for Parallel Programming for Machine Learning 2025-2026.

1. <u>Abstract</u>

• Brief problem description
This project focuses on counting **bigrams** (pairs of consecutive bytes) in a piece of text. The idea is simple: given a large text, we want to scan it and compute how often each possible bigram appears. The sequential version works well but takes time when the input is large, so **parallelizing** it makes sense.

• Approach summary
I split the input into several chunks so each thread works on a different part of the text. Each thread keeps its own local histogram, which avoids race conditions. At the end, I just merge all of them into the final histogram.

• Key results (best speedup achieved)
The parallel version does improve the runtime compared to the sequential one. The best speedup achieved was around **1.244×** using **4** threads.

2. <u>Introduction</u>

• Problem description and motivation
The goal of this project is to count bigrams, which are simply pairs of consecutive bytes in a text. Even though the operation is straightforward, bigram counting is useful in many areas such as **text analysis**, **compression**, and **simple language models**.

• Computational challenges
The main challenge is that all threads need to **update** the same histogram of 65,536 counters, and writing to shared memory can easily create **race conditions**. Another challenge is splitting the data so that each thread works on an independent region without needing extra synchronization. Finally, after all threads finish their part, the results need to be combined correctly, which adds some overhead.

• Objectives and scope
The main objectives of the project are:

1. Implement a **clean and correct** sequential version of the bigram counter.
2. Identify the parts of the algorithm that can be **parallelized** safely.
3. Develop a parallel implementation using **OpenMP**, avoiding race conditions and unnecessary synchronization.
4. Run experiments with different numbers of threads and measure performance properly (multiple runs, warm-ups, statistics, etc.).
5. Analyze the results and discuss why the **speedups** behave the way they do.

The **scope** of the project focuses only on the **computation** itself (not on improving I/O or memory usage). The idea is mainly to see how much speedup we can get from parallelizing such a simple loop and what limitations show up.

3. Algorithm Description

• Sequential algorithm explanation

The sequential version of the bigram counter is very simple. First, the whole input file is loaded into a vector of bytes. Then, the program initializes a **histogram** with 65,536 positions (256 × 256), one for each possible bigram. The algorithm loops through the text from the first byte to the second-to-last one, reads the pair (s[i], s[i+1]), computes its index in the histogram, and increments the corresponding counter.

In pseudocode, the core of the algorithm looks like this:

```
for i = 0 to n-2:
a = s[i]
b = [i + 1]
histogram[a*256 + b]++
```

• Identification of parallelizable sections

The main loop that scans the text can be parallelized easily because each iteration works on a **different pair of bytes**. In other words, the count for position i is completely **independent** from the count at i+1, so each thread can work on its own chunk of the input without needing to coordinate during the counting phase.

Thanks to this, we can **parallelize** the **loop** that goes through the text and updates a **local histogram** in each thread and the final **merge** of the **partial histograms**, since every position of the histogram can be combined independently.

4. Parallel Implementation

• Parallelization strategy

The program divides the input text into several equal **chunks** so each thread processes a different part of the array. Every thread has its own private histogram of 65,536 counters to avoid race conditions. Once all threads finish, their **local histograms are combined** into the final global one in a separate merge step.

• OpenMP directives used

The implementation uses:

```
#pragma omp parallel num_threads(T)
```
to create the thread team.

```
#pragma omp for schedule(static)
```
to split the iterations of the counting loop evenly.

Another parallel for `#pragma omp for schedule(static)` for the merge of the partial histograms.

Static scheduling works well here because each iteration does the same amount of work.

• Synchronization mechanisms

No explicit synchronization is needed during the counting phase, since each thread writes only to **its own histogram**. The only synchronization comes from the implicit barriers at the end of the omp for loops. No atomics or locks were required.

• Data structures and memory layout

The histograms are stored as **flat arrays** for simplicity: one global histogram, and a vector of per-thread local histograms. This makes the indexing easier and helps avoid false sharing.

• Challenges and solutions

There were a few challenges found, like **race conditions** which were solved, as already mentioned, by giving each thread its own histogram, **efficient merging**, solved by parallelizing the merge loop and **having a stable timing**. For this last one, I had to create a constant `static const int REPEATS = 150;` in order to repeat the computation many times so each run produces a more stable time, and then multiple external runs are used to compute statistics.

5. <u>Experimental Setup</u>

• Hardware specifications

All experiments were run on my personal laptop. The relevant specs are:
CPU: AMD Ryzen 7 6800H
Cores / Threads: 8 cores / 16 threads
RAM: 16 GB
Cache: 512 KB L1, 4 MB L2, 16 MB L3
OS: Windows 11 (64-bit)

• Software environment

Compiler: Microsoft Visual Studio 2022 (MSVC)
Standard: C++17
OpenMP: Enabled via Visual Studio's project configuration
Build type: Release mode (O2/O3 optimization)
Libraries: Only standard C++ and OpenMP; no external dependencies.

• Dataset used

Since the parallel version needs a large enough input to show meaningful differences, I generated a text file of around 100 MB. The program simply reads the file as raw bytes, so no preprocessing or tokenization was required.

• Parameters tested

I tested different numbers of threads to see how the parallel version scales:
1, 2, 4, 8, 16 threads

```
int num_threads = 16;  // other possibilities that were tried are 1,2,4,8,16
```

• Measurement methodology

Before taking any measurements, I run the computation **twice as warm-up** to avoid cold-start effects. Since the work done in a single pass of the bigram counter is very small compared to the precision of the system timer, I repeat the computation 150 times inside the program (REPEATS = 150). This makes the total measured time large enough to be stable and easier to compare across **different thread configurations**.

Then, for each thread configuration I **run the program multiple times** (5-10) and record the resulting times (including sequential and parallel time, and speedup). From these runs I compute the required statistics (mean, standard deviation, minimum and maximum).

Only the **computation phase** is timed. File loading and initialization happen once and are excluded from measurement.

6. Results and Analysis

• Correctness validation

To verify correctness, I compared the output of the sequential implementation with the parallel one after each test. Both versions produced the exact same histogram. This check is done automatically in the code by comparing the two vectors.

```
// 3) Validation: check if both vectors are the same
if (counts_seq == counts_par) {
    cout << "VALIDATION OK: secuential y parallel match.\n";
}
else {
    cout << "ERROR: secuential and parallel result are not the same.\n";
}
```

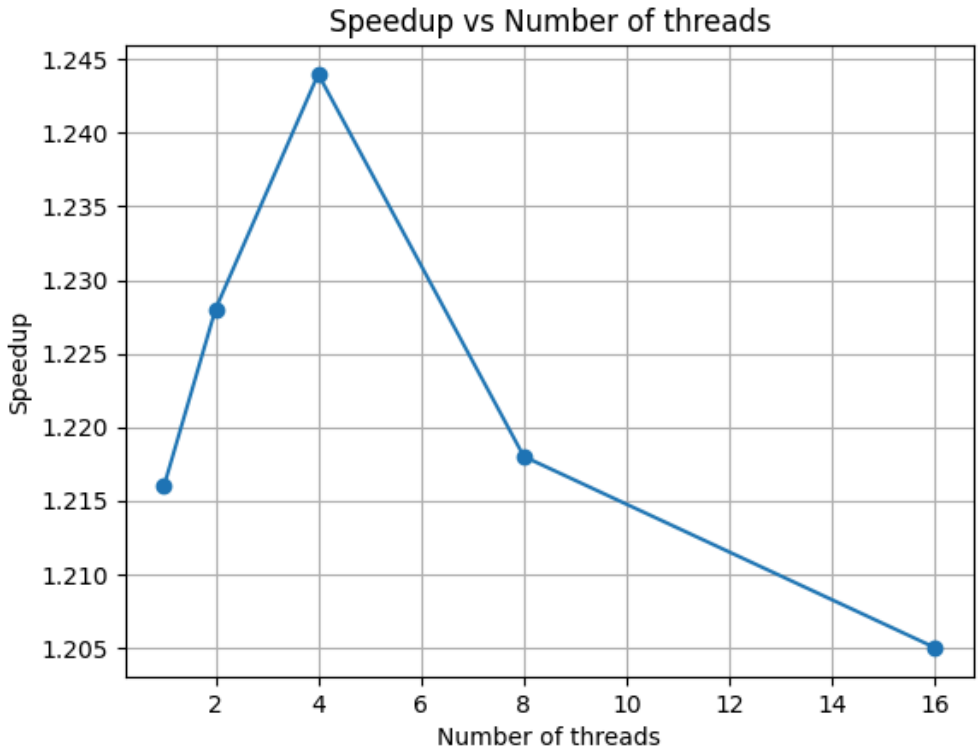
• Performance results:

For each thread configuration (1, 2, 4, 8, 16), after 2 warm-up executions, I ran the program multiple times and computed the **mean**, **standard deviation**, **minimum, maximum** and the average **Speedup** (Sequential/Parallelized time). The results below correspond to the average time per execution (after REPEATS = 150 inside the program).

Results

Threads	#	Mean (ms)	Std Dev (ms)	Min	Max	Speedup
1		59.466	1.012	57.87	60.51	1.216×
2		58.504	1.272	57.06	60.26	1.228×
4		57.480	0,233	57.26	57.85	1.244×
8		58.846	0,766	58.00	59.68	1.218×
16		59.294	0,507	58.43	59.74	1.205×

• Speedup curves (speedup vs threads)
This is the representation of the **Speedup** compared to the **Number of threads**:



We observe that the speedup peaks at 4 threads and decreases afterwards. This drop is primarily due to the **overhead of managing additional threads**, **memory bandwidth saturation**, and the **merge phase cost,** which outweigh the benefits of parallelization beyond this point. These bottlenecks are analyzed in detail in the 'Bottleneck analysis' section.

• Scalability analysis (strong/weak scaling)
Since the dataset is fixed (100 MB input text), this corresponds to **strong scaling**.
Observations:
The counting loop itself scales well because each thread works **independently**.
The merge step (65,536 entries × number of threads) **limits ideal scaling** slightly.
The CPU has 8 physical cores, so using more than 8 threads does not lead to significant speedup.
Overall, the parallel version does scale, but not perfectly, which is expected for this type of task.

• Effect of different parameters (scheduling, chunk size, etc.)
I used **schedule(static)** for the main loop because each iteration does the same fixed amount of work. I also tried different scheduling strategies like dynamic but it did not show any improvement, since the workload is **perfectly balanced by default.**

Chunk-size changes were also unnecessary because **OpenMP's default block splitting** already matches the structure of the data and produces my desired results.

• Comparison of variants: different parallelization strategies

I evaluated the impact of different OpenMP scheduling clauses, specifically comparing **schedule(static)** versus **schedule(dynamic)**. The experiments confirmed that **static scheduling** was the **superior** choice for this algorithm. Since the computational cost per byte is constant, the workload is naturally balanced. Consequently, using **schedule(dynamic) was counterproductive**; it introduced unnecessary runtime overhead for managing task chunks without providing any load-balancing advantage.

• Bottleneck analysis

At first, the improvement was not the one I expected, but then I understood that there were some factors that were limiting scalability:

Memory bandwidth: all threads read the same large array, so bandwidth is shared.

Merge overhead: although parallelized, the merge step still requires reading T (number of threads histograms.

Small amount of work per iteration: each iteration does very little work (two loads + one increment), so the loop becomes **memory-bound** rather than compute-bound.

```
for i = 0 to n-2:
a = s[i]
b = [i + 1]
histogram[a*256 + b]++
```

These bottlenecks explain why speedups are visible but not dramatic.

7. <u>Discussion</u>

• Interpretation of results

The results show that the performance only improves slightly when increasing the number of threads. The average speedup stays around 1.20–1.25 for all configurations, and in some cases it even gets worse when using 8 or 16 threads.

This means that the parallel version works correctly, but the algorithm is not compute-heavy enough to benefit from many threads.

The execution is dominated by memory access, not by computation as already explained.

• Comparison with theoretical expectations (Amdahl's Law, Gustafson's Law)

According to **Amdahl's Law**, the maximum speedup is limited by the fraction of work that cannot be parallelized.

Here, reading the file, allocating structures, merging histograms, and the thread management overhead all contribute to this serial part. This explains why the **speedup never gets close to the ideal value**.

According to **Gustafson's Law**, scalability can improve with larger problem sizes. Even though my dataset was already large, the algorithm still does **very little work** per iteration, so increasing the input size does not drastically change the nature of the bottleneck. The results are consistent with a memory-bound workload rather than a compute-bound one.

• Overhead analysis
Several overheads affect performance:

Thread creation and synchronization: even if OpenMP is efficient, creating many threads for a very simple loop introduces some cost.
Memory bandwidth pressure: all threads read from the same large array, which saturates the memory subsystem.
Merge phase: even though it is parallelized, merging thousands of counters adds extra work.
Low work per iteration: each iteration only performs a couple of operations, so threads spend more time waiting for memory than doing actual work.
These overheads explain why performance stops improving beyond 4 threads.

• Limitations and constraints
The main limitations observed are similar to the already mentioned, which of course, makes a lot of sense because it shows that is all related within:
The algorithm is **heavily limited** by **memory bandwidth**.
My CPU has a **limited number of physical cores**, so using more threads than cores adds contention.
The amount of **per-iteration computation is very small.**
The **parallel version** performs **more total work** than the sequential one (due to local histograms and merging).

• Lessons learned
During this project I learned many things, I can highlight a few:
Parallelizing code **does not** automatically **guarantee large speedups**.
Memory bottlenecks often dominate in simple workloads.
Using more threads than necessary can slow things down.
Warm-up runs and **repeated measurements** are important to obtain stable results.
Validation between sequential and parallel versions is **essential** before analyzing performance.

• Conclusions
The OpenMP parallelization works correctly and the **results match t**he sequential version.
However, the **performance improvement is modest** due to the memory-bound nature of the problem and the low computational intensity per iteration.
The **best speedup** obtained was around **1.244×** using **4 threads**, but adding more threads provided no benefit and sometimes slightly worsened performance.
This shows that not all algorithms scale well, even if they look easy to parallelize.

• Summary of achievements
Implemented both **sequential** and **parallel versions** of the bigram counter.
Ensured correctness with full validation.
Designed a **thread-safe solution** using local histograms.
Parallelized the merge phase.

Built a **measurement system** with warm-up runs and multiple repetitions.
Collected and analyzed performance data.
Produced the **final speedup curve**.

• Best configuration found
The best configuration found, on average, was using **4 threads**, with a **mean of 57.48 ms** and an **average speedup of 1.244x** but the one that got the best singular result was using **2 threads**, with **57.06 ms of execution time** and a **speedup of 1.254x**.

• Future work suggestions
For future work, one of the best suggestions is to **close as many programs** that may be running in the back and that make the execution less stable. I noticed that when I had other programs running in the background, I obtained much more different results which did not really help me evaluate the optimization of parallelizing.
Also try to be in the **same conditions to record the results**, like battery, efficient or gaming mode, etc…

8. <u>References</u>

• Academic papers, documentation, dataset sources
My main documentation was the one on the slides on this course, and as referenced before, I also used python to create the graphic to show the curve, and finally, I made some references to the code, to relate directly to the code and the explanation.